Motor gráfico con Python de cuerpos en presencia de un campo de fuerzas

¿Cómo simular y visualizar situaciones físicas en el seno de un campo de fuerzas?

XXXIV Certamen Jóvenes Investigadores 2023
convocado por el Instituto de la Juventud y el Ministerio de Universidades

Investigación en el ámbito de las asignaturas de Matemáticas II, Física y TIC de 2ºBachillerato.
Curso 2021/2022

COLEGIO MARISTA "LA INMACULADA" DE GRANADA

19 de enero de 2023

Autor del trabajo (curso 2021/2022):
Hugo Pérez Muñoz (2ºBachillerato)
Profesor coordinador del trabajo (curso 2021/2022):
Daniel Partal García

Motor gráfico con Python de cuerpos en presencia de un campo de fuerzas

¿Cómo simular y visualizar situaciones físicas en el seno de un campo de fuerzas?

Índice

RESUMEN 2
ABSTRACT 2
1. INTRODUCCIÓN 3
2. ANTECEDENTES 4
3. HIPÓTESIS DEL TRABAJO Y OBJETIVOS DE LA INVESTIGACIÓN 5
3.1 ¿POR QUÉ ELEGIR PYTHON? 5
3.2 ¿POR QUÉ UTILIZAR JSON? 5
4. MATERIALES Y MÉTODOS 5
4.1 CÓDIGO PYTHON Y JSON 6
4.2 ESTRUCTURA DE DATOS DEL CÓDIGO PYTHON 6
4.2 REPRESENTACIÓN GRÁFICA EN TIEMPO REAL 6
4.3 ECUACIONES FÍSICAS EN LA DETERMINACIÓN DE LOS CAMPOS 7
4.4 COLISIÓN ENTRE DOS CUERPOS 8
4.5 CONSERVACIÓN DE LA ENERGÍA MECÁNICA 9
4.6 MÉTODO RANGE-KUTTA EN LA APROXIMACIÓN DE INTEGRALES 9
4.7 MENÚ INICIAL PARA LA SELECCIÓN DE SIMULACIONES Y MODIFICACIÓN DE PARÁMETROS. 9
4.8 INTELIGENCIA ARTIFICIAL DE ALGORITMO GENÉTICO 10
5. RESULTADOS 10
5.1 EL PROBLEMA DE LOS TRES CUERPOS 11
5.2 EL PROBLEMA DE N-CUERPOS 11
5.3 SISTEMA SOLAR 11
5.4 CARGA ELÉCTRICA EN MOVIMIENTO DENTRO DE CAMPO MAGNÉTICO GENERADO POR UN CONDUCTOR RECTILÍNEO 11
6. CONCLUSIONES 11
6.1 FORTALEZAS Y DEBILIDADES DE NUESTRO MODELO 12
7. AGRADECIMIENTOS 12
ANEXOS DE FIGURAS 13
BIBLIOGRAFÍA Y WEBGRAFÍA 17

RESUMEN

La Física es la interpretación y comprensión de la naturaleza a partir de fundamentos matemáticos.

Visualizar, en tiempo real en el espacio tridimensional, la evolución de las ecuaciones matemáticas que rigen el movimiento de un cuerpo (o varios cuerpos) en el seno de un campo gravitatorio, eléctrico o magnético es fundamental en el proceso de enseñanza-aprendizaje de esta rama de la ciencia. Pero la complejidad de la simulación en un intervalo de tiempo provoca que, con frecuencia, la docencia de la Física en Bachillerato se vea limitada al estudio de aproximaciones sencillas o al estudio de los instantes iniciales del movimiento.

El presente proyecto busca solventar esta tradicional laguna en la enseñanza de las ciencias en niveles preuniversitarios, brindando un motor gráfico informático bajo código Python con el que poder reproducir, en tres dimensiones, movimientos de objetos afectados por las ecuaciones de la ley de la gravitación universal, la ley de Coulomb y la ley de Lorentz.

Las principales dificultades que superar en la implementación del motor son la representación del movimiento en tres dimensiones en una pantalla bidimensional, y el tamaño del incremento temporal de una iteración a la siguiente, ya que un salto temporal grande provoca simulaciones irreales y un salto temporal muy pequeño consume enormes cantidades de recursos de computación.

Creemos que nuestra propuesta solventa con garantías estos retos y ofrece un simulador útil y didáctico para cualquier persona interesada en las leyes de la Física.

ABSTRACT

Physics is the interpretation and comprehension of nature through a mathematical base.

To visualize, in real time and in 3D, how the mathematical ecuations that set a body's movement (or multiple bodies) in a gravitatory, electric or magnetic field change is necessary whether for learning or teaching this branch of sciencie. However, the complexity of the simulation in a time interval usually limits teaching in bachelor's degrees to the study of simple aproximations or only the movement's starting phases.

The current project aims to reduce this barrier in the teaching of science in pre-universitary levels, providing a Python physics engine capable of reproducing, graphically in 3D, the movements of bodies unde the influence of the ecuations of the law of universal gravitation, Coulomb's law and Lorentz's law.

The main problems to solve in the design of the engine are the representation of 3D movements in a 2D screen, the size of time steps between frames, as a big time step produces simulations that aren't a faithful representation of reality and steps too small require huge computational work.

To our minds this approach can solve these challenges and provides an useful and educational engine for anyone interested in the Physics' laws.

1. INTRODUCCIÓN

Un cuerpo que se desplaza en línea recta, desde una posición inicial s_0 respecto al origen del sistema de referencia, con una velocidad inicial constante v_0 y bajo la influencia de un sistemas de fuerzas que generan una aceleración uniforme a, alcanza una posición final s_f dependiente del tiempo.

$$s_f(t) = s_0 + v_0 \cdot \Delta t + \frac{1}{2} \cdot a \cdot (\Delta t)^2 \quad \textbf{(1)}$$

Donde Δt representa el intervalo temporal medido desde el inicio del movimiento hasta el final. Esta ecuación rige el desplazamiento del conocido MRUA (movimiento rectilínea uniformemente acelerado), objeto de estudio en la asignatura de Física de la Enseñanza Secundaria Obligatoria (ESO).

En movimientos horizontales, el vector velocidad con sentido hacia la derecha conlleva un valor positivo de v_0 en la **ecuación (1)**, mientras que el vector velocidad con sentido hacia la izquierda provoca un valor negativo. En movimientos verticales, el sentido ascendente de la velocidad es positivo y el sentido descendente es negativo.

Si la aceleración favorece el sentido del movimiento, es positiva (genera un aumento en el módulo de la velocidad). Y si la aceleración se opone al sentido del movimiento, es negativa (aceleración de frenado).

Cuando los alumnos progresan en su destrezas con las matemáticas, gracias a la trigonometría y al concepto de vector, dan el paso en 4ºESO y en 1ºBachillerato a la descripción del movimiento bidimensional parabólico.

Este movimiento parabólico, asumiendo la aproximación de ausencia de rozamiento con la atmósfera, describe el desplazamiento horizontal y vertical de un cuerpo que parte de una posición inicial $s_0(x_0, y_0)$ con sendas componentes cartesianas, tras el impulso inicial representado por un vector velocidad $\vec{v}$ que forma un ángulo α con el eje de abscisas y que ve frenado su ascenso vertical por la acción descendente de la fuerza gravitatoria terrestre, caracterizada por el vector vertical de aceleración $\vec{g}$.

Descomponiendo el movimiento parabólico en un movimiento rectilíneo (MRU) horizontal y en un MRUA vertical, podemos parametrizar la posición del cuerpo en función del tiempo.

$$\begin{cases} x(t) = x_0 + |\vec{v}| \cdot \cos(\alpha) \cdot \Delta t \\ y(t) = y_0 + |\vec{v}| \cdot \text{sen}(\alpha) \cdot \Delta t - \frac{1}{2} \cdot |\vec{g}| \cdot (\Delta t)^2 \end{cases} \quad \textbf{(2)}$$

Asumimos que la aceleración gravitatoria se opone en todo momento al ascenso vertical de la segunda coordenada cartesiana, por lo cual aparece el signo negativo en la contribución de la aceleración en la **ecuación (2)**.

Esta limitación del estudio a los casos de una y dos dimensiones se extrapola a los siguientes contenidos propios de la asignatura de Física en Bachillerato:

- Ley de gravitación universal de Newton, aproximando planetas y estrellas a cuerpos puntuales que se mueven en un plano bidimensional, con uno de los cuerpos siempre en reposo.
- Ley del campo eléctrico de Coulomb, con una distribución de cargas coplanarias. También es bastante común asumir posiciones iniciales estáticas para las cargas eléctricas.
- Campo magnético, asumiendo un hilo conductor unidimensional infinito al cual se le acerca una carga eléctrica con un determinado ángulo de inclinación, que facilita la reducción del estudio del movimiento al caso bidimensional.

En las imágenes proporcionadas por libros de texto y webs de referencia, el movimiento del objeto se simula como una superposición de fotogramas que describen, poco a poco, su trayectoria **(Figura 1 de Anexos)**. Pero cabría preguntarse:

- ¿Qué ocurre si los efectos de la aceleración cambian de manera dinámica, debido al movimiento propio de los cuerpos a lo largo del tiempo?
- ¿Cómo asumir un vector aceleración no uniforme en las ecuaciones del movimiento?
- ¿Cómo afecta la presencia de n-cuerpos en los problemas que tradicionalmente se reducen al estudio de solo dos cuerpos?
- ¿Cómo visualizar el movimiento en el espacio tridimensional?
- ¿Qué ocurre si todos los cuerpos de la simulación parten con un vector velocidad tridimensional no nulo?

Estas preguntas iniciales marcan los retos a solventar por nuestro motor gráfico con Python.

2. ANTECEDENTES

La Ley Orgánica de Modificación de la LOE (LOMLE) es el marco legal educativo vigente en España, desde 2020, para las enseñanzas regladas preuniversitarias. Su ordenación y enseñanzas mínimas para Bachillerato están publicadas en el Real Decreto 243/2022 del 5 de abril[1].

Es una ley competencial, es decir, busca que el alumno alcance las destrezas suficientes para solventar situaciones de la vida cotidiana en los que deba expresarse con fluidez (en su lengua materna y en una lengua extranjera), aplicar razonamientos lógico-matemáticos en el contexto de las ciencias y la tecnología, manejar entornos digitales, con capacidad de autoaprendizaje, de trabajo en equipo, con iniciativa emprendedora y valorando el entorno cultural a su disposición en la sociedad.

El proyecto presentado en este informe encaja perfectamente con el espíritu y con la norma educativa actual, ya que permite al alumno trabajar descriptores operativos íntimamente relacionados con la adquisición óptima de las competencias clave:

- Se expresa con fluidez para intercambiar información y crear conocimiento con sus compañeros, sus profesores y con la comunidad de desarrolladores Python en internet.
- Localiza, analiza y selecciona información de diferentes fuentes.
- Utiliza el inglés en la lectura de obras técnicas y en la participación de diversos foros online.
- Utiliza los métodos inductivos y deductivos propios del razonamiento matemático.
- Desarrolla el método científico en la creación de conocimiento, estableciendo hipótesis a verificar o refutar.
- Diseña prototipos y simulaciones, que son mejoradas gracias a la revisión constructiva de profesores y compañeros de clase.
- Aporta soluciones digitales y tecnológicas a necesidades concretas planteadas en las asignaturas de Matemáticas II, Física y TIC de 2ºBachillerato.
- Aplica criterios de fiabilidad a la información publicada en internet, manejándose con seguridad y respetando los derechos de autor de los algoritmos publicados en diversos foros técnicos sobre Python.
- Ofrece las conclusiones de su solución tecnológica al resto de compañeros de clase, que mejoran en su proceso de enseñanza-aprendizaje gracias a las simulaciones creadas por el motor gráfico.
- Crece en su aprendizaje de manera autónoma, más allá de los contenidos propios del Bachillerato incluso para el caso de alumnos brillantes.
- Planifica su trabajo y lo ejecuta de manera ordenada en un periodo largo de investigación.
- Evalúa necesidades y afronta retos.
- Promueve soluciones innovadoras, aplicando conocimientos técnicos específicos.

El uso de software matemático para la realización de modelados de situaciones fisicomatemáticas es una realidad ya consolidada en los institutos de Secundaria y Bachillerato de España. Los programas de referencia se simulación (con suscripciones de pago) como Matlab[2] y Mathematica[3] han dado paso, gracias a la contribución de desarrolladores de código abierto, a una infinidad de potentes alternativas gratuitas a disposición de alumnos y docentes de cualquier nivel educativo.

Podríamos destacar en la actualidad Geogebra[4] (fruto de la tesis doctoral del austriaco Markus Hohenwarter en 2002), programa matemático de uso libre que aúna la geometría dinámica y el cálculo simbólico. Cuenta con un potente visor gráfico en dos y tres dimensiones, con multitud de funciones matemáticas ya prediseñadas y una buena comunidad de desarrolladores distribuidos por todo el mundo. Es multiplataforma y sus creaciones son fácilmente publicables en entornos web.

Como limitaciones de Geogebra podríamos señalar que la consola disponible para la introducción de código de programación propio (por ejemplo, en JavaScript o en el lenguaje nativo de Geogebra) es poco amigable y que el uso de recursos de memoria de la computadora es tan elevado que, cuando las simulaciones se complican por el número de objetos que aparecen o por el incremento de iteraciones, puede generar en el bloqueo de un ordenador de nivel medio-alto.

Sería deseable una interfaz especialmente pensada para la introducción de código de programación masivo, como ya existen para lenguajes de rápida interpretación por la computadora como son C, C++, JAVA, etc. Pero estas interfaces no poseen un motor gráfico con el que visualizar el movimiento de los objetos en las simulaciones, por lo que asumimos que conseguirlo con Python[5] será una reto de una magnitud considerable.

¡Manos a la obra!

3. HIPÓTESIS DEL TRABAJO Y OBJETIVOS DE LA INVESTIGACIÓN

El objetivo principal de nuestro proyecto es desarrollar un motor gráfico de situaciones físicas, bajo lenguaje Python, con el objetivo de que el algoritmo y su visualización se ejecuten rápido en un ordenador de nivel medio.

Asumimos las siguientes hipótesis iniciales de trabajo en el desarrollo del motor gráfico:

- Entendemos el tiempo como una sucesión de pequeños incrementos, con el objeto de simular el movimiento de los cuerpos de la manera más realista posible. El tamaño de estos incrementos vendrá determinado por la capacidad del ordenador para mostrar de manera fluida los fotogramas asociados a cada incremento.
- Cuando surjan colisiones entre los cuerpos de la simulación, asumiremos colisiones elásticas ideales. No hay deformación en los cuerpos y se conserva la cantidad de movimiento.
- Además de visualizar la posición de los cuerpos, en cada incremento temporal, la interfaz gráfica debe informar de manera continua de los módulos de las velocidades y de las magnitudes de las unidades físicas más relevantes en la simulación (posición, masa, carga eléctrica).
- Con objeto de medir la precisión de los cálculos del motor gráfico, calcularemos la energía mecánica total del sistema de cuerpos cuando los campos sean conservativos.
- La interfaz debe proporcionar la posibilidad de aplicar escalas para mejorar la visualización de los cuerpos en las simulaciones.
- Para facilitar la portabilidad de las simulaciones entre dispositivos, el programa debe permitir guardar la información de toda la simulación en archivos independientes.
- Un menú interactivo permitirá al usuario la creación de cuerpos, campos y la modificación de parámetros de la simulación (constante gravitatoria, constante eléctrica y permeabilidad magnética en el vacío).
- Inicialmente, proponemos aplicar una escala lineal y logarítmica en la representación. La escala logarítmica estaría especialmente pensada para simulaciones con distancias entre cuerpos de varios órdenes de magnitud.
- Asumimos que la forma de los cuerpos es esférica.

3.1 ¿Por qué elegir Python?

Python es un lenguaje de programación sencillo de aprender y aplicar, con la posibilidad de crear programas complejos y profesionales en un tiempo de desarrollo no demasiado amplio.

Además, Python posee una gran comunidad de usuarios online que comparten códigos y librerías de manera altruista. Esto ayuda a localizar soluciones concretas a muchos de los retos que van planteándose a lo largo del proyecto. Es un lenguaje multiplataforma, lo que refuerza el carácter educativo del proyecto, ya que permite al usuario ejecutar y modificar el programa desde cualquier dispositivo.

Es más lento en su ejecución que otros lenguajes de alto nivel (como C o C++), pero sus bibliotecas gráficas (como por ejemplo Pygame[6]) son una gran ayuda en el desarrollo de la aplicación y permiten optimizar esfuerzos. Asumimos que la velocidad de ejecución con Python no tendrá una repercusión crítica sobre el coste computacional en un ordenador de nivel medio.

3.2 ¿Por qué utilizar JSON?

JSON[7] nos permite almacenar las simulaciones en archivos de texto que se pueden leer y editar manualmente. Además, es muy sencillo importar las simulaciones de JSON a Python y viceversa. Es independiente del lenguaje de programación empleado. Convirtiéndolo en una opción ideal para el intercambio de datos. Los archivos quedan almacenados con una extensión .json y se pueden abrir, incluso, con un editor de texto.

Para facilitar la experiencia del usuario con nuestro proyecto, no solo nos limitamos a tener una base de datos de simulaciones en JSON sino que también ofrecemos un menú interactivo que facilita el uso del simulador para cualquier tipo de usuarios (alumnos de Secundaria, alumnos de Bachillerato, docentes o personas en general interesadas en las simulaciones físicas).

4. MATERIALES Y MÉTODOS

Los archivos .py y .json con todo el código de programación del proyecto, y los vídeos de las simulaciones de ejemplo, se pueden descargar en la siguiente carpeta de Google Drive:

https://drive.google.com/drive/folders/1ftvTpXoDriAnoBIjB6xkNGCGqWBZkas9?usp=share_link

4.1 Código Python y JSON

El proyecto se desarrolla bajo código Python. El lenguaje JSON se centra para guardar y cargar las simulaciones como una herramienta de almacenamiento de datos.

El código Python realiza todos los cálculos y representaciones tridimensionales en tiempo real, permitiendo:

- Seleccionar los campos de fuerza requeridos en la simulación: gravitatorio, eléctrico uniforme, magnético uniforme, magnético generado por un conductor rectilíneo infinito. No hay limitación en el número ni el tipo de campos que se pueden incluir en la simulación.
- De cada campo, el usuario puede modificar parámetros como la dirección, la intensidad. En el campo magnético asociado al conductor rectilíneo se puede modificar la posición del hilo conductor.
- Visualizar las líneas de campo entrantes y salientes en campos uniformes. Visualizar el conductor rectilíneo indefinido.
- Añadir una cantidad indefinida de cuerpos, editando parámetros como la posición inicial, la velocidad inicial, la aceleración inicial, la masa o la carga. En las magnitudes vectoriales se puede seleccionar la orientación del vector. Como es lógico, la capacidad de computación del ordenador del usuario marcará el número máximo de cuerpos a simular.
- Configurar la velocidad de la simulación, pausarla, guardarla, cargarla, cambiar el zoom en tiempo real o moverse a través del espacio.
- Visualizar los datos de los cuerpos en tiempo real.

Los datos almacenados en un diccionario de Python se pueden trasladar directamente, sin conversión, a un fichero JSON donde se accede a la información a través de un campo llave **(Figura 2 de Anexos)**.

4.2 Estructura de datos del código Python

Por complejidad ascendente, los tipos de datos que hemos diseñado para la construcción del motor físico son:

- **Vectores.** Incluyen operaciones básicas como la suma, la multiplicación de escalar por vector, el producto escalar y el producto vectorial, obtener el vector unitario equivalente, calcular su módulo o calcular el ángulo entre dos vectores.
- **Rastros.** Conjunto de puntos por los que ha pasado un cuerpo. Tiene un tamaño máximo y un color asociados.
- **Cuerpos, Campos y Cámara**. Tanto los cuerpos como la cámara poseen una masa, posición, velocidad y aceleración. Por un lado, los cuerpos tienen carga, una imagen asociada y un radio. Por otro lado, la cámara tiene un zoom y una velocidad de zoom. Los campos almacenan una variable que determina su tipo (gravitatorio, eléctrico, magnético), su intensidad, dirección y datos estéticos. Además, los campos magnéticos se diferencian entre uniformes o generados por un conductor rectilíneo. En el caso de los conductores, se almacena también un punto por el que pase el conductor.
- **Entornos.** Un entorno es un conjunto de cuerpos y campos que parten de unas condiciones iniciales. Cada entorno tiene su propia configuración (cantidad de cálculos por unidad de tiempo, orden de Range-Kutta, duración, dimensiones de la ventana, etc.), que determinará cómo calculará la posición de los cuerpos transcurrido un breve delta time (Δt). Un entorno se puede guardar en cualquier momento de la simulación, sabiendo que se sobrescribirán las condiciones iniciales. Se puede cargar o crear de cero desde el menú. Los entornos también pueden calcular la energía mecánica del sistema en un momento determinado, herramienta que nos será útil para medir su precisión.

4.2 Representación gráfica en tiempo real

Dada una situación inicial, buscamos calcular cómo evolucionan los cuerpos después de un pequeño intervalo de tiempo (delta time[8]).

Si la aceleración sufrida por la interacción con los campos fuera constante aplicaríamos directamente las ecuaciones del MRUA para el espacio tridimensional. Mostraríamos en pantalla la nueva situación e iteraríamos el proceso hasta que el usuario detuviera la simulación. Si la aceleración cambia de manera dinámica, este proceso se complica.

Para representar la simulación tridimensional en una pantalla bidimensional, debemos implementar un motor gráfico que proyecte cada elemento de la simulación sobre la pantalla en función de la posición de un "observador" o cámara. Esto es computacionalmente bastante exigente y difícil de implementar, cuyo proceso podemos simplificar con el siguiente razonamiento.

Solo nos interesa mostrar visualmente el comportamiento de los cuerpos bajo influencias de diferentes campos. Asumimos que todos los cuerpos de la simulación tendrán forma esférica, por lo que su visualización no depende de la perspectiva. Con esta hipótesis de trabajo inicial, es posible reflejar en la pantalla la tridimensionalidad de la simulación reduciendo el tamaño de los cuerpos según su distancia a la cámara[9].

Una vez creado el motor físico que permite simular virtualmente la evolución de un conjunto de cuerpos, dadas unas condiciones iniciales, avanzamos pequeños saltos en el tiempo (Δt) y mostramos una nueva imagen para el nuevo estado del sistema. Repetimos hasta que el usuario finaliza la simulación.

¿Qué podemos hacer mientras se ejecuta la simulación?

- (P) Pausar la simulación.
- (Q) Salir de la simulación.
- (I) Mostrar / ocultar información de los cuerpos.
- (O) Mostrar / ocultar rastros de los cuerpos.
- (R) Reiniciar la simulación.
- (Con la rueda del ratón) Aumentar o disminuir el zoom.
- (W,A,S,D, Z, X) Mover la cámara por el espacio, respectivamente (Arriba, Izquierda, Abajo, Derecha, hacia dentro, hacia fuera).
- (+, -) Aumentar o disminuir la velocidad de la simulación, es decir, incrementar o reducir la cantidad de cálculos por unidad de tiempo.

4.3 Ecuaciones físicas en la determinación de los campos

La Ley de Gravitación Universal[10] enunciada por Newton en 1687 describe cómo se ve afectado el estado de movimiento de dos cuerpos debido a la interacción gravitatoria entre sus masas. Dado un cuerpo con una masa m_1, en una posición P_1, y un segundo cuerpo con una masa m_2, en una posición P_2, la ley de gravitación propone que la fuerza que genera el primero sobre el segundo viene dada por la siguiente ecuación:

$$\vec{F}_{1-2} = -G \cdot \frac{m_1 \cdot m_2}{r^2} \cdot \hat{\mu} \quad \textbf{(3)}$$

Dónde G es la constante de gravitación universal ($6{,}67428 \cdot 10^{-11}\,\frac{m^3}{kg}$). El valor r es la distancia entre los dos cuerpos, y $\hat{\mu}$ es el vector unitario que apunta de P_1 a P_2. El signo menos representa que la fuerza gravitatoria es atractiva.

De forma equivalente, la Ley de Coulomb[11] explica cómo se ve afectado el estado de movimiento de dos cuerpos debido a la interacción electrostática entre sus cargas. Dado un cuerpo con una carga c_1, en una posición P_1, y un segundo cuerpo con una carga c_2, en una posición P_2, la ley de Coulumb nos dice que la fuerza que genera el primero sobre el segundo viene dada por la siguiente ecuación:

$$\vec{F}_{1-2} = K \cdot \frac{c_1 \cdot c_2}{r^2} \cdot \hat{\mu} \quad \textbf{(4)}$$

Dónde K es la constante eléctrica del medio, igual a $8{,}98755 \cdot 10^{-9}\,\frac{N \cdot m^2}{C^2}$ en el vacío. De nuevo r es la distancia entre los dos cuerpos, y $\hat{\mu}$ es el vector unitario que apunta de P_1 a P_2. Si la fuerza es atractiva o repulsiva dependerá del signo de las cargas. Dos cargas del mismo signo creará una fuerza repulsiva, mientras que dos cargas de distinto signo generan una fuerza atractiva.

La Ley de Biot y Savart[12] muestra el campo magnético $\vec{B}$, medido en Teslas, generado por un conductor rectilíneo de longitud l, por el que circula una intensidad $\vec{I}$, en un punto P. Sin embargo, y dado que nosotros solo consideramos conductores rectilíneos infinitos debido a que son más simples, utilizaremos la ecuación que describe el campo generado por dichos conductores:

$$\vec{B} = \mu \cdot \frac{\vec{I} \times \hat{r}}{2\pi r} \quad \textbf{(5)}$$

Siendo μ la constante de permeabilidad magnética del medio, equivalente a $4\pi \cdot 10^{-7}\,\frac{T \cdot m}{A}$, $\hat{r}$ el vector unitario que une el conductor con el punto, y r la distancia desde el punto al conductor.

La Ley de Lorentz[13] nos ayudará a determinar cómo variará el estado de movimiento de un cuerpo con carga q, con una velocidad $\vec{v}$ bajo la influencia de un campo magnético $\vec{B}$:

$$\vec{F}_m = q \cdot \vec{v} \times \vec{B} \quad \textbf{(6)}$$

En el caso del campo eléctrico, la fuerza que sufre una carga q bajo el efecto de campo eléctrico $\vec{E}$ se define como:

$$\vec{F} = q \cdot \vec{E} \quad \textbf{(7)}$$

De forma similar, la fuerza que sufre un cuerpo de masa m dentro de un campo gravitatorio $\vec{g}$ es:

$$\vec{F} = m \cdot \vec{g} \quad \textbf{(8)}$$

Estas ecuaciones nos permitirán obtener las aceleraciones sufridas por los cuerpos dentro de dichos campos de fuerza uniformes.

4.4 Colisión entre dos cuerpos

Por la Ley de conservación de la cantidad movimiento[14], sabemos que en un sistema de partículas la cantidad de movimiento total se mantiene constante si no interactúan fuerzas externas. Si consideramos un sistema formado por dos partículas con velocidades iniciales, que terminan chocando entre sí, podemos ayudarnos de esta ley para calcular cuál será la velocidad de cada cuerpo después del choque.

La cantidad de movimiento $\vec{p}$ de una partícula viene dada por el producto de su masa y velocidad:

$$\vec{p} = m \cdot \vec{v} \quad \textbf{(9)}$$

De esta forma, la cantidad de movimiento inicial del sistema formado por dos cuerpos será igual al sumatorio de la cantidad de movimiento de cada uno de los cuerpos:

$$\overrightarrow{p_1} + \overrightarrow{p_2} = m_1 \cdot \overrightarrow{v_1} + m_2 \cdot \overrightarrow{v_2} \quad \textbf{(10)}$$

Además, siempre que no intervengan fuerzas no conservativas, la energía mecánica del sistema también se mantendrá constante, de acuerdo con la Ley de conservación de la energía mecánica. En nuestro sistema inicial de dos cuerpos planteado, solo estamos considerando inicialmente la velocidad de los cuerpos, por lo que la energía mecánica se calcula directamente como la suma de las energías cinéticas de ambos objetos:

$$E_m = \frac{1}{2} m_1 \cdot |\overrightarrow{v_1}| + \frac{1}{2} m_2 \cdot |\overrightarrow{v_2}| \quad \textbf{(11)}$$

Así, podemos plantear un sistema de dos ecuaciones con dos incógnitas si añadimos los valores de la cantidad de movimiento final y la energía mecánica final, dado que las masas de los cuerpos también se mantienen constantes:

$$\begin{cases} \overrightarrow{p_0} = \overrightarrow{p_f} \\ E_{m_0} = E_{m_f} \end{cases} \Rightarrow \begin{cases} m_1 \cdot \overrightarrow{v_{1_o}} + m_2 \cdot \overrightarrow{v_{2_o}} = m_1 \cdot \overrightarrow{v_{1_f}} + m_2 \cdot \overrightarrow{v_{2_f}} \\ \frac{1}{2} m_1 \cdot |\overrightarrow{v_{1_o}}| + \frac{1}{2} m_2 \cdot |\overrightarrow{v_{2_o}}| = \frac{1}{2} m_1 \cdot |\overrightarrow{v_{1_f}}| + \frac{1}{2} m_2 \cdot |\overrightarrow{v_{2_f}}| \end{cases} \quad \textbf{(12)}$$

Despejando las velocidades finales de cada uno de los cuerpos:

$$\begin{cases} \overrightarrow{v_{1_f}} = \frac{(m_1 - m_2) \cdot \overrightarrow{v_{1_o}} + 2m_2 \cdot \overrightarrow{v_{2_o}}}{(m_1 + m_2)} \\ \overrightarrow{v_{2_f}} = \frac{(m_2 - m_1) \cdot \overrightarrow{v_{2_o}} + 2m_1 \cdot \overrightarrow{v_{1_o}}}{(m_1 + m_2)} \end{cases} \quad \textbf{(13)}$$

Obtenemos la expresión que usará nuestro motor gráfico el programa para determinar la velocidad de los cuerpos tras su colisión.

Gracias a que suponemos todos los cuerpos esféricos, detectar las colisiones entre ellos se limita a comprobar si la distancia entre ambos es mayor o menor a la suma de sus radios. El radio de un cuerpo viene determinado por el tamaño de la imagen que tiene asociada.

La comprobación de colisión se realiza después de desplazar a los cuerpos. Esto tiene dos consecuencias principales sobre el curso de la simulación.

La primera consecuencia: si un cuerpo va lo suficientemente rápido con respecto a otro puede “atravesarlo”, y como después del desplazamiento ya no se encuentra en colisión, seguiría su movimiento sin verse afectado por el choque. Para las simulaciones que realizaremos con nuestro motor gráfico, los cuerpos no son tan pequeños ni viajan tan rápido respecto a los saltos temporales delta time como para atravesarse unos a otros de forma que la imprecisión del

método sea notable a nivel visual. Por lo que entendemos que nuestra forma de proceder no entorpece el carácter didáctico de la simulación.

La segunda consecuencia: idealmente, en el momento del choque, la separación entre los cuerpos sería igual a la suma de sus radios. Pero en la práctica de la simulación, tras el desplazamiento previo a la detección de la colisión, encontraremos que las partículas estarán ligeramente alejadas de esta situación ideal. Esto nos obliga a mover los cuerpos hasta que queden tangentes, antes de ejecutar el choque entre ellos. Esto produce imprecisiones, especialmente a la hora de conservar la energía mecánica del sistema, pues estamos "teletransportando" cuerpos para marcar las condiciones iniciales de la colisión.

4.5 Conservación de la energía mecánica

La energía mecánica de los cuerpo se compone de energía cinética y potencial:

$$E_m = E_c + E_p \quad \textbf{(14)}$$

La energía cinética de un cuerpo depende de su masa y del cuadrado de su velocidad. La energía potencial, en simulaciones con campos no uniformes, se descompone a su vez en energía potencial gravitatoria y energía potencial eléctrica (sabiendo que la energía potencial es la integral de la fuerza sobre el tiempo)

$$E_c = \frac{1}{2} \cdot m \cdot \overrightarrow{|v|}^2$$

$$E_p = E_{p_g} + E_{p_e} = -G \cdot \frac{Mm}{r} + K \cdot \frac{Qq}{r} \quad \textbf{(15)}$$

Para cada cuerpo, calculamos su energía cinética y la añadimos al total. Luego, calculamos su energía potencial respecto del resto de cuerpos y añadimos la mitad de dicha energía potencial. Añadimos solo la mitad mitad ya que, si A tiene una energía potencial respecto de B, entonces B tiene la misma energía potencial respecto de A, debido a que la fuerza que sienten ambos cuerpos es la misma (ley de acción y reacción). El resultado de todas esas sumas será la energía mecánica del sistema.

4.6 Método Range-Kutta en la aproximación de integrales

El método Range-Kutta[15] es un método numérico de aproximación que permite obtener una solución a ecuaciones integrales cuando podemos evaluar la función a integrar. Nuestro interés principal es utilizar el método para obtener la posición de un cuerpo transcurrido un periodo de tiempo Δt, dada su aceleración k_1 en un instante inicial.

Suponiendo esa aceleración inicial constante, calculamos la posición que alcanzaría el cuerpo si transcurriera la mitad del tiempo $\frac{\Delta t}{2}$. En dicha posición, se calcula la nueva aceleración que sufriría (k_2).

Volvemos a operar para obtener la posición en la que terminaría el cuerpo si partiera desde la posición inicial con dicha aceleración k_2 durante la mitad del salto de tiempo. De nuevo, se calcula la aceleración en dicho punto (k_3) y, para terminar, operamos para obtener la aceleración que sufriría el cuerpo si desde la posición inicial partiera con esa aceleración durante el periodo de tiempo completo (k_4).

Con las diferentes aceleraciones, del método Range-Kutta, la velocidad final tras transcurrir el intervalo temporal dependerá de la media ponderada de dichas aceleraciones, según la expresión:

$$v_{n+1} = v_n + \frac{1}{6} \cdot \Delta t \cdot (\, k_1 + 2\, k_2 + 2\, k_3 + \, k_4) \quad \textbf{(16)}$$

Este método proporciona la velocidad del cuerpo transcurrido un tiempo Δt, y podríamos volver a aplicarlo utilizando el mismo algoritmo para calcular la posición de manera más precisa[16] **(Figura 3 de Anexos)**.

Tras cada movimiento/paso de Range-Kutta hacemos una comprobación de colisiones.

4.7 Menú inicial para la selección de simulaciones y modificación de parámetros

El menú inicial **(Figura 4 de Anexos)** permite:

- Iniciar, modificar, crear y eliminar simulaciones.
- Añadir, modificar, copiar y eliminar cuerpos y campos de fuerza.
- Configurar la visualización.

- Ajustar los valores de la constante de gravitación universal, eléctrica y permeabilidad magnética.
- Determinar la velocidad de simulación, el orden de Range-Kutta, la duración de la simulación y las dimensiones de la ventana donde se visualiza.
- A cada cuerpo se le puede asociar la posición, la velocidad y la aceleración iniciales, la masa, la carga eléctrica y una imagen (teniendo en cuenta que el cuerpo siempre es esférico).
- En cada campo de fuerza podemos definir el tipo de campo (gravitatorio, eléctrico o magnético). En el caso del magnético se puede elegir entre uniforme o generado por conductor rectilíneo infinito. También podemos establecer la intensidad del campo y su dirección. En el conductor rectilíneo podemos ajustar su posición.

4.8 Inteligencia artificial de algoritmo genético

Los algoritmos genéticos se basan en las Leyes de la Vida Natural descritas por Darwin en 1859. Partimos de un conjunto de individuos con características propias y una función para evaluar su capacidad de reproducción. Para problemas de optimización, partimos de una población inicial de un tamaño determinado con características aleatorias, evaluamos a los individuos y seleccionamos qué individuos dejarán descendencia (a mayor puntuación obtenida en la evaluación, mayores posibilidades de ser seleccionado).

Cada individuo está compuesto por cromosomas que contienen los genes que codifican sus características. Cuando un individuo es seleccionado para dejar descendencia, se crea un entorno idéntico, y se le somete a un proceso de mutación. En el proceso de mutación cada uno de los genes tiene una probabilidad determinada de variar, permitiendo que los hijos sean ligeramente distintos a los padres, pudiendo explorar diferentes combinaciones de parámetros.

Mediante esta técnica proponemos la resolución de problemas que carecen de ecuaciones que las resuelvan[17]. Como por ejemplo: dadas tres cargas que son lanzadas con una determinada velocidad inicial, ¿cuál debe de ser la velocidad inicial de la tercera carga para que acabe en una posición determinado transcurrido un intervalo temporal concreto?

En nuestro proyecto, los individuos serán entornos de simulación y sus genes codificaran las propiedades de los cuerpos y los campos que albergan. Para evaluarlos iniciaremos la simulación desde las condiciones iniciales hasta el periodo de tiempo indicado, y compararemos la posición y la velocidad de los cuerpos solicitados con los valores deseados. La suma de las diferencias entre los resultados deseados y los obtenidos conformarán el error de una simulación, y su puntuación equivaldrá al inverso de dicho error:

$$puntuación = \frac{1}{error} \quad \textbf{(17)}$$

Una vez puntuados los entornos, los "hijos" resultantes del proceso tendrán opción solo a mutar las características que se indiquen en el problema. En el ejemplo expuesto anteriormente sobre las tres cargas, solo se podría mutar la velocidad del tercer cuerpo.

5. RESULTADOS

Recordamos que **los archivos .py y .json con todo el código de programación del proyecto, y los vídeos de las simulaciones de ejemplo, se pueden descargar en la siguiente carpeta de Google Drive:**

https://drive.google.com/drive/folders/1ftvTpXoDriAnoBIjB6xkNGCGqWBZkas9?usp=share_link

Hemos logrado crear un motor físico capaz de representar la esencia de la evolución de un sistema de cuerpos y campos a lo largo del tiempo. No requiere de un excesivo poder computacional ni de conocimientos de programación. Con su bajo consumo, su diseño en Python y su menú interactivo, está al alcance de todo tipo de profesores y alumnos interesados.

Somos conscientes de lo ineficiente que es utilizar Python para las simulaciones más intensivas, como sería necesario en la Inteligencia Artificial. El hecho de que sea un lenguaje semi interpretado implica que ha de traducir cada instrucción a lenguaje máquina en tiempo real. Además, no permite el uso del procesamiento multihilo, que consiste en ejecutar varias instrucciones de un proceso simultáneamente. Solo permite el multiprocesamiento, que implica crear un nuevo proceso por cada conjunto de instrucciones que se quiere realizar de forma paralela. La creación de un hilo es considerablemente más rápida a la de un proceso, y todos los hilos comparten la memoria del proceso al que pertenecen, mientras que en el multiprocesamiento cada proceso ha de estar cargado en memoria principal.

Con unos errores de conservación de la energía mecánica inapreciables a simple vista (**ver ejemplo del vídeo disponible en el enlace indicado a Google Drive - Figura 5 de Anexos**), del orden de las milésimas, concluimos en que nuestro motor de físico simula de forma similar a la que sucedería en la realidad.

5.1 El problema de los tres cuerpos

Dados tres cuerpos, sus posiciones y velocidades iniciales, el motor de físicas es capaz de ir determinando cómo evoluciona el sistema a lo largo del tiempo. En el **ejemplo del vídeo disponible en el enlace indicado a Google Drive (Figura 6 de Anexos)**, las masas no son realistas, la constante gravitatoria vale 1 (para acelerar los cálculos) y los saltos de tiempo se cuentan por segundos (delta time no es una fracción de segundo sino un valor de varios segundos), ya que al ser masas muy pequeñas respecto a las distancia que de separación avanzan muy lentamente.

En la simulación se aprecia que los cuerpos se hacen más grandes o pequeños conforme se acercan o se alejan de la pantalla. También se pueden observar las colisiones elásticas, y con ellas una de las limitaciones de nuestra simulación. Al aplicar la perspectiva no hemos conseguido que se "deforme" visualmente el espacio y las imágenes de los cuerpos de la misma forma. Esto produce que la visualización no sea del todo fiel al entorno que simula.

5.2 El problema de n-cuerpos

Si ya era imposible cuadrar con ecuaciones el estado de un sistema con tres cuerpos, hacerlo para n-cuerpos nos adentramos en un problema matemático inimaginable. No obstante, para nuestro motor de físico es tan sencillo como añadir los cuerpos e incluirlos en los cálculos de cada iteración (**ver ejemplo del vídeo disponible en el enlace indicado a Google Drive - Figura 7 de Anexos**).

5.3 Sistema Solar

El Sistema Solar es uno de los problemas de n-cuerpos más antiguos de la historia. En nuestro proyecto mostramos cómo los planetas se desenvuelven a lo largo del tiempo.

Hemos obtenido los datos de todos los planetas y del Sol por medio de la web de la NASA llamada Horizons System[18]. Dado un punto de referencia y una fecha, imprime los datos de un cuerpo (masa, radio, posición, velocidad relativa, etc). El tiempo de la simulación se mide en segundos, aunque hemos de tener en cuenta que la reducción del radio tendrá una repercusión sobre el periodo orbital de los cuerpos (**ver ejemplo del vídeo disponible en el enlace indicado a Google Drive - Figura 8 de Anexos**).

5.4 Carga eléctrica en movimiento dentro de campo magnético generado por un conductor rectilíneo

Dada una carga con una velocidad inicial paralela a la dirección de un conductor rectilíneo por el que circula una intensidad, el motor nos muestra como la carga describe un movimiento helicoidal que envuelve al conductor (**ver ejemplo del vídeo disponible en el enlace indicado a Google Drive - Figura 9 de Anexos**).

6. CONCLUSIONES

El uso de escalas lineales para mostrar simulaciones tan grandes como las del sistema solar es ineficiente, ya que las distancias de los planetas al Sol difieren en diferentes órdenes de magnitud, lo que dificulta considerablemente la visualización, incluso después de aplicar la escala.

Se nos ocurrió utilizar una escala logarítmica, que reduciría dichas distancias de diferentes órdenes a unas cantidades mucho más fáciles de mostrar en pantalla. Sin embargo, aunque la escala funciona muy bien cuando los valores absolutos de las coordenadas de los cuerpos son muy grandes, produce un movimiento demasiado rápido cuando las coordenadas son pequeñas.

Otra forma de acortar distancias que también ahorre bastante trabajo computacional, consiste en aplicar una reducción a las masas de todos los cuerpos y calcular a qué distancia deberíamos situar los cuerpos para que estos sintieran la misma fuerza gravitatoria a pesar de la reducción en la masa.

$$\overrightarrow{|F_g|} = G \cdot \frac{Mm}{r^2}$$

$$\overrightarrow{|F_g|}' = G \cdot \frac{M'm'}{r'^2}$$

$$\overrightarrow{|F_g|} = \overrightarrow{|F_g|}' \Longrightarrow G \cdot \frac{Mm}{r^2} = G \cdot \frac{M'm'}{r'^2} \Rightarrow r'^2 \cdot Mm = r^2 \cdot M'm'$$

Teniendo en cuenta que las nuevas masas son iguales a las anteriores, multiplicadas por una constante k de reducción:

$$r'^2 \cdot Mm = r^2 \cdot k^2 \cdot M \cdot m \Rightarrow r' = r \cdot k \quad \textbf{(18)}$$

Con lo que concluimos que, para actualizar la distancia de forma que la fuerza gravitatoria no se vea afectada por la reducción en la masa, basta con multiplicar la distancia por ese mismo facto de reducción. Esto facilitará la visualización y, además, aumentará la velocidad de los cálculos gracias a la simplificación de los números.

6.1 Fortalezas y debilidades de nuestro modelo

Nuestras simulaciones poseen las siguientes fortalezas:

- Proyecto Multiplataforma que muestra con gran fidelidad física entornos matemáticamente muy complejos.
- Facilidad de uso, sin necesidad de grandes destrezas digitales.
- Proyecto de código abierto que permite la mejora de otros usuarios.
- Las simulaciones generadas son un magnífico recurso didáctico en la enseñanza de Física.
- Flexibilidad a la hora de crear simulaciones con elevado número de cuerpos y de campos de fuerza.
- Posibilidad de ampliación con nuevas fuerzas o interacciones.

Las debilidades de nuestro proyecto son:

- El uso de Python implica un tiempo de ejecución en la computadora elevado y, además, dificulta el multiprocesamiento. Podríamos cambiar de lenguaje para reducir el tiempo de computación (por ejemplo, C o C++) y maximizar así los resultados de la Inteligencia Artificial.
- En el paso de la aceleración al espacio debemos integrar dos veces. Eso implica un doble proceso Range-Kutta. En primer lugar, para pasar de la aceleración a la velocidad. Y luego para pasar de la velocidad al espacio. La complejidad de los cálculos asciende, por el doble proceso, a $O(n^2)$ (siendo n el orden de Range-Kutta, que en nuestro proyecto ha sido 4). Esto provoca una reducción en la velocidad de simulación o bien un mayor error en los cálculos.
- Mejorar la visualización de las colisiones y la capacidad de rotar la cámara en el espacio (por ejemplo, proyectando los cuerpos sobre un plano respecto a la cámara).
- El sistema de colisiones tiene un amplio margen de mejora a través de la implementación de técnicas más modernas de detección, así como la búsqueda lineal o la búsqueda binaria[19].
- El algoritmo genético también tiene mucho margen de mejora, principalmente en la creación de nuevos individuos. Nuestro algoritmo solo permite mutar componentes de los genes. Un algoritmo más completo también cruzaría a los individuos de la población, creando hijos a través de recombinaciones de ADN de los supervivientes.
- A la hora del cálculo intensivo es común ayudarse de la tarjeta gráfica de la computadora para realizar operaciones. Otra propuesta de mejora es mejorar el código para que haga el máximo uso posible del ordenador coordinando CPU y GPU.

7. AGRADECIMIENTOS

Gracias al claustro de profesores (no solo de las asignaturas vinculadas de Matemáticas II, Física y TIC) por cedernos ciertos huecos horarios para el desarrollo del proyecto.

Gracias a los compañeros de 2ºBachillerato Tecnológico del curso 2021-2022 por su revisión constante en pro de la mejora del proyecto.

Gracias a Rebeca Victoria Herrero Hahn, del Departamento de Electrónica y Tecnología de Computadores, por sus recomendaciones para solventar el problema de escalas en simulaciones con grandes distancias entre cuerpos.

El reto acometido en el presente proyecto ha sido, sin duda alguna, apasionante, instructivo y alentador para continuar con futuras investigaciones a nivel universitario.

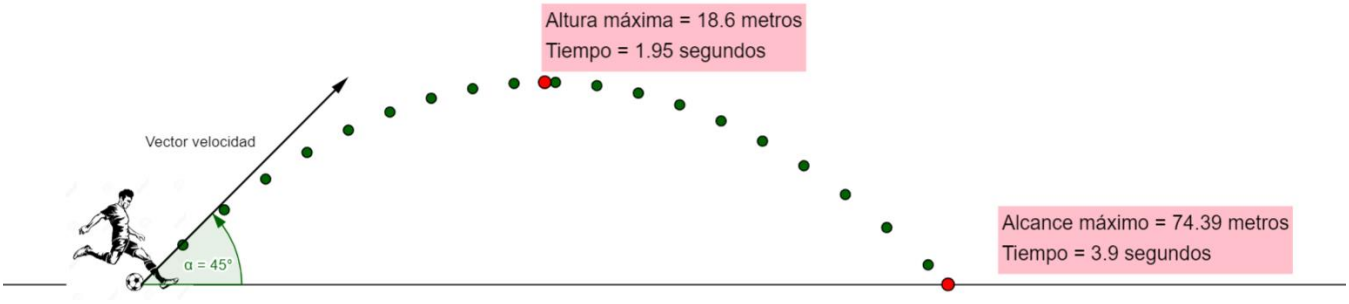


Figura 1. Representación clásica del estudio del movimiento parabólico como la unión de diferentes fotogramas temporales. Figura de elaboración propia.

Simulaciones > carga y conductor > Cuerpos > carga > {} datos.json > # Carga

```
{"Posici\u00f3n": [0.0, 100.0, 0.0],
"Velocidad": [20.0, 0.0, 0.0],
"Aceleraci\u00f3n": [0.0, 0.0, 0.0],
"Masa": 5.0,
"Carga": 1.0}
```

Figura 2. Código JSON que almacena los datos de un cuerpo en un instante determinado de la simulación.

```
def Simular(self, deltatime, range_kutta=1):
    coeficientes = []
    denominador = 0
    for k in range(range_kutta // 2):
        coeficientes.append(2**k)
        denominador += 2**k
    if (range_kutta % 2 == 1):
        coeficientes.append(2**(range_kutta // 2 + 1))
        denominador += 2**(range_kutta // 2 + 1)

    for k in range(range_kutta // 2):
        coeficientes.append(2**(range_kutta // 2 - k))
        denominador += 2**(range_kutta // 2 - k)

    for nombre in self.cuerpos:
        cuerpo = self.cuerpos[nombre]

        cuerpo.velocidad_inicial = cuerpo.velocidad
        cuerpo.posicion_inicial = cuerpo.posicion

    for k in range(range_kutta - 1):
        #Calculamos la aceleración y guardamos la posición y velocidad iniciales:
        self.CalcularAceleracion()

        for nombre in self.cuerpos:
            cuerpo = self.cuerpos[nombre]
            cuerpo.velocidad = cuerpo.velocidad_inicial + cuerpo.aceleracion * deltatime #Euler
            cuerpo.kutta_v.append(cuerpo.velocidad)

            #Movemos los cuerpos a la mitad del desplazamiento producido por dicha aceleración: (mid point)
            cuerpo.posicion = cuerpo.posicion_inicial + 0.5 * cuerpo.velocidad_inicial * deltatime + 0.25 * cuerpo.aceleracion * (deltatime / 2)**2

    for nombre in self.cuerpos:
        cuerpo = self.cuerpos[nombre]
        cuerpo.velocidad = cuerpo.velocidad_inicial + cuerpo.aceleracion * deltatime
        cuerpo.kutta_v.append(cuerpo.velocidad)

        cuerpo.posicion = cuerpo.posicion_inicial
        for k in range(range_kutta):
            cuerpo.posicion += (coeficientes[k] * deltatime * cuerpo.kutta_v[k] / denominador)

        cuerpo.kutta_v = []
        self.Colisiones()
```

Figura 3. Detalle del código que aplica el método de Range-Kutta dado un orden (en nuestro proyecto hemos aplicado hasta orden 4). Figura de elaboración propia.

```
Bienvenido al menú principal. Selecciona una opción:
1) Ver simulaciones disponibles
2) Iniciar una simulación
3) Crear una nueva simulación
4) Copiar una simulación
5) Editar una simulación existente
6) Borrar una simulación
v) Volver atrás.
q) Salir del menú.
Escribe tu elección:
```

Figura 4. Menú inicial de la simulación. Figura de elaboración propia.

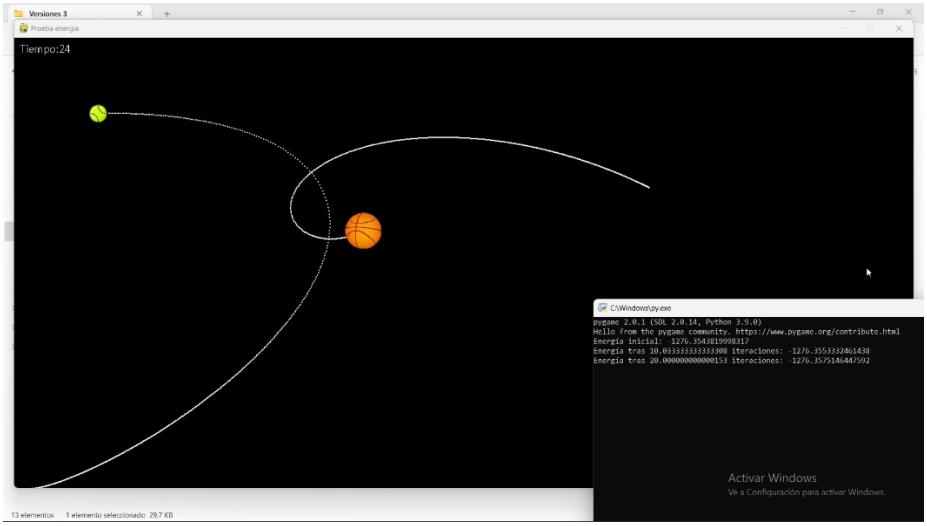


Figura 5. Los cálculos del motor gráfico muestran una conservación de la energía mecánica del entorno del orden de las milésimas, lo que ratifica la solidez de nuestras simulaciones. Figura de elaboración propia. Enlace a vídeo de la simulación: https://drive.google.com/drive/folders/1ftvTpXoDriAnoBIjB6xkNGCGqWBZkas9?usp=share_link

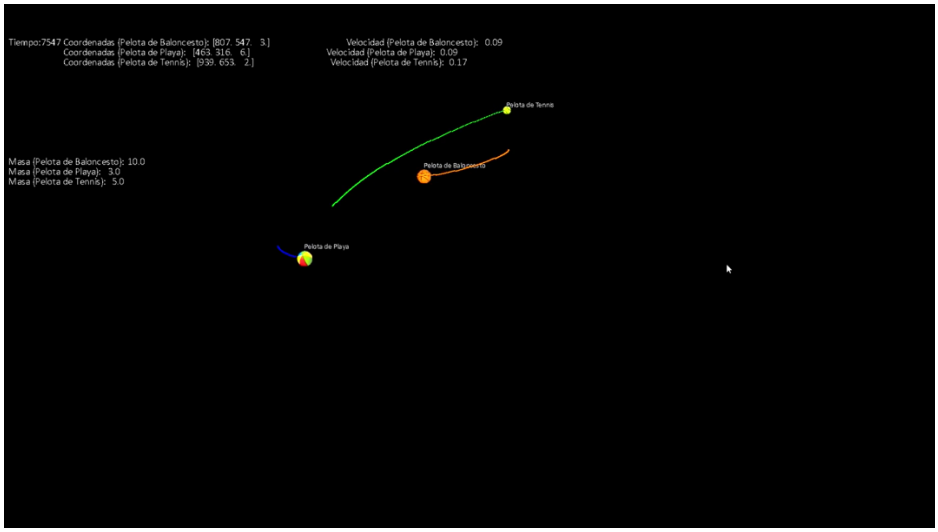


Figura 6. Captura de pantalla de una simulación con 3 cuerpos. Figura de elaboración propia. Enlace a vídeo de la simulación: https://drive.google.com/drive/folders/1ftvTpXoDriAnoBIjB6xkNGCGqWBZkas9?usp=share_link

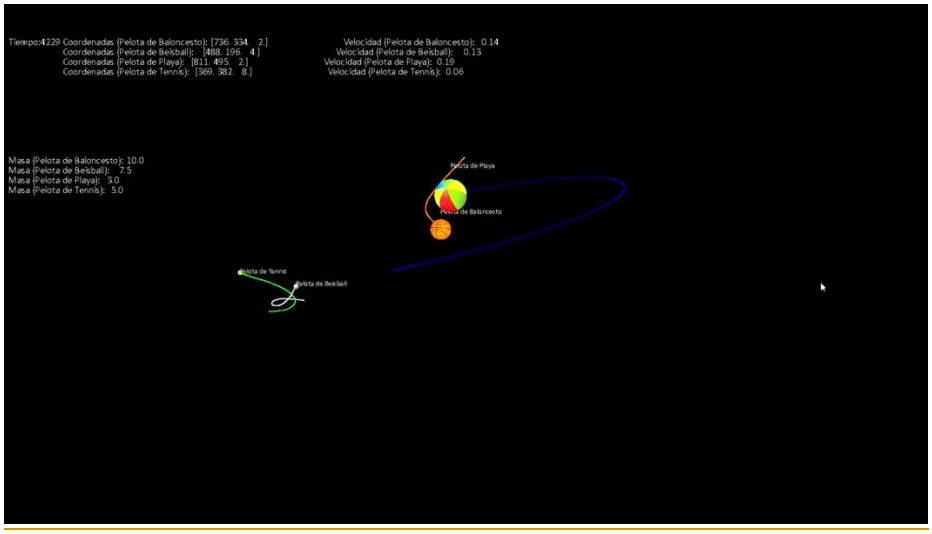


Figura 7. Captura de pantalla de una simulación con n cuerpos. Figura de elaboración propia. Enlace a vídeo de la simulación: https://drive.google.com/drive/folders/1ftvTpXoDriAnoBIjB6xkNGCGqWBZkas9?usp=share_link

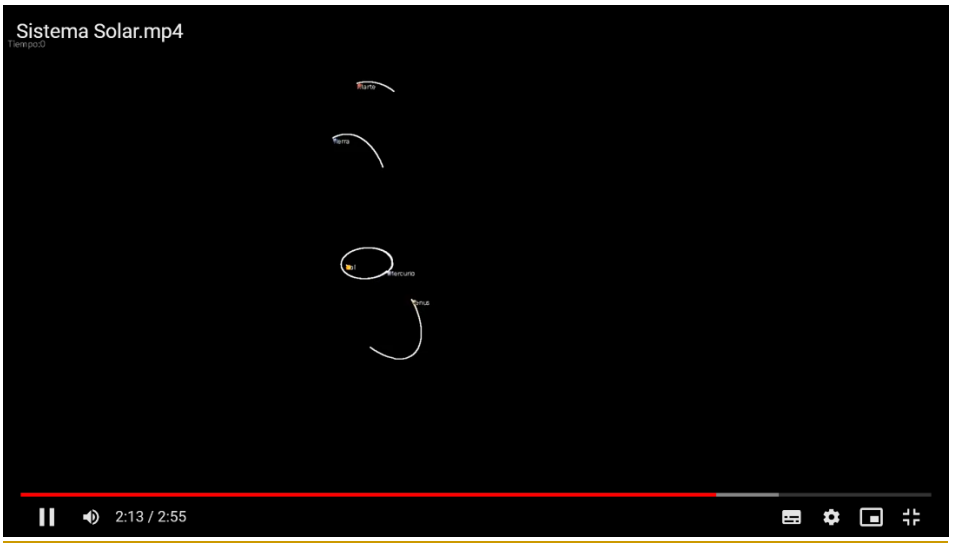


Figura 8. Captura de pantalla de una simulación del Sistema Solar. Figura de elaboración propia. Enlace a vídeo de la simulación: https://drive.google.com/drive/folders/1ftvTpXoDriAnoBIjB6xkNGCGqWBZkas9?usp=share_link

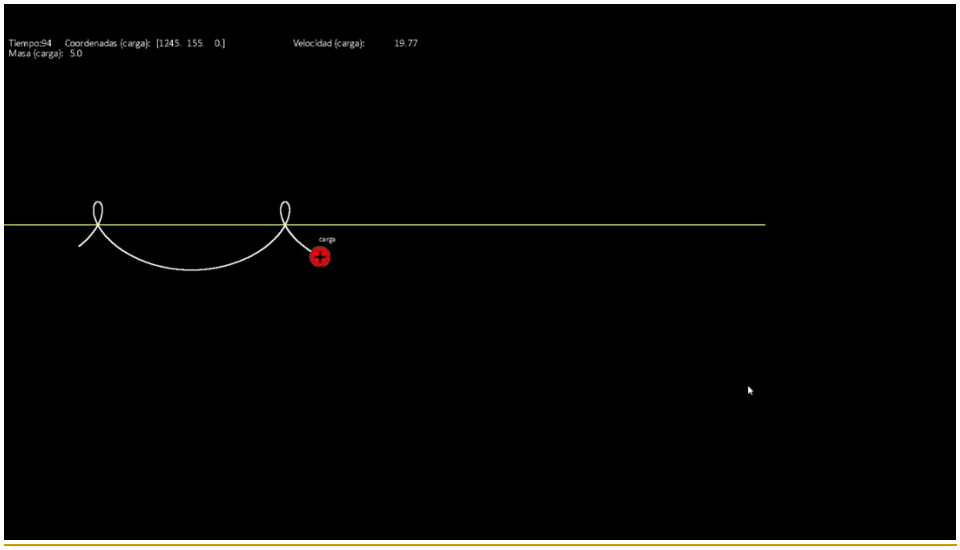


Figura 9. Captura de pantalla de una simulación de carga eléctrica en el seno de campo magnético generado por un hilo indefinido. Figura de elaboración propia. Enlace a vídeo de la simulación: https://drive.google.com/drive/folders/1ftvTpXoDriAnoBIjB6xkNGCGqWBZkas9?usp=share_link

BIBLIOGRAFÍA Y WEBGRAFÍA

[1] Gobierno de España. Ministerio de Educación y Formación Profesional. 2020. *Real Decreto 243/2022, de 5 de abril, por el que se establecen la ordenación y las enseñanzas mínimas del Bachillerato*. [en línea]
Available at: https://www.boe.es/buscar/act.php?id=BOE-A-2022-5521
[Último acceso: 18 Febrero 2022]

[2] Matlab. Plataforma de programación y cálculo numérico. [en línea]
Available at: https://es.mathworks.com/products/matlab.html
[Último acceso: 20 marzo 2022]

[3] Mathematica. Sistema de computación técnica moderna. [en línea]
Available at: https://www.wolfram.com/mathematica
[Último acceso: 20 marzo 2022]

[4] Geogebra. Herramientas digitales gratuitas para enseñar y aprender Matemáticas. [en línea]
Available at: https://www.geogebra.org
[Último acceso: 20 marzo 2022]

[5] Python España. [en línea]
Available at: https://es.python.org
[Último acceso: 15 mayo 2022]

[6] Pygame documentation. [en línea]
Available at: https://www.pygame.org/docs
[Último acceso: 18 mayo 2022]

[7] JSON JavaScript Object Notation. [en línea]
Available at: https://www.json.org/json-es.html
[Último acceso: 17 abril 2022]

[8] ¿Por qué utilizar Delta Time? [en línea]
Available: https://www.parallelcube.com/es/2017/10/25/por-que-necesitamos-utilizar-delta-time
[Último acceso: 18 mayo 2022]

[9] Simulación 2D en Python. [en línea]
Available at: https://www.youtube.com/watch?v=ErA-lRBAhTk
[Último acceso: 4 marzo 2022]

[10] Ley de gravitación universal. Universidad de Cantabria. [en línea]
Available at: https://personales.unican.es/mierja/docencia/educacion/master/fps/apuntes/17_18/EXPERIMENTOS-HISTORIA-FISICA-2018-2017-03-MECANICA-R-PARTE-02.pdf
[Último acceso: 5 abril 2022]

[11] Ley de Coulomb. Universidad de Málaga. [en línea]
Available at: https://ocw.uma.es/pluginfile.php/937/mod_resource/content/0/tema1_01_doc.pdf
[Último acceso: 7 abril 2022]

[12] Ley de Biot y Savart. Universidad de Málaga. [en línea]
Available at:
https://ocw.uma.es/pluginfile.php/883/mod_resource/content/0/Tema05_Fuentes_C_magnetico_2011_OCW.pdf#:~:text=Ley%20de%20Biot%20y%20Savart&text=Christian%20Oersted%20observ%C3%B3%20que%20la,las%20proximidades%20del%20conduc%2D%20tor.
[Último acceso: 7 abril 2022]

[13] Ley de Lorentz. Universidad de Sevilla. [en línea]
Available at: http://laplace.us.es/campos/teoria/grupo2/tema6.pdf
[Último acceso: 7 abril 2022]

[14] Ley de conservación de la cantidad de movimiento. Universidad de Sevilla. [en línea]
Available at: http://tesla.us.es/wiki/images/d/d4/MR_Tema06_1718.pdf
[Último acceso: 8 abril 2022]

[15] Métodos de Range-Kutta. [en línea]
Available at: https://www.mathstools.com/section/main/Metodos_de_Runge_Kutta?lang=es#.Y8mBj3bMI2w
Available at: https://www.youtube.com/watch?v=hGCP6I2WisM&t=437s
[Último acceso: 4 marzo 2022]

[16] Ejemplo práctico de aplicación del método Range-Kutta en aproximación de integrales. Tesina de Ezequiel Moreno. Universidad Nacional de la Plata (Argentina). [en línea]
Available at: http://sedici.unlp.edu.ar/bitstream/handle/10915/103363/Documento_completo.pdf-PDFA.pdf?sequence=1&isAllowed=y
[Último acceso: 4 marzo 2022]

[17] Proyecto fin de carrera de José Carlos López Díaz sobre algoritmo genético en transformaciones lineales. Departamento de Informática. Universidad Carlos III de Madrid. [en línea]
Available at: https://core.ac.uk/reader/29401985
[Último acceso: 15 enero 2023]

[18] Horizons System. [en línea]
Available at: https://ssd.jpl.nasa.gov/horizons/app.html#/
[Último acceso: 15 enero 2023]

[19] Teoría de colisiones 2D: Conceptos básicos. [en línea]
Available at: https://www.genbeta.com/desarrollo/teoria-de-colisiones-2d-conceptos-basicos
[Último acceso: 15 enero 2023]